A Monodromy Computer by Path Lifting

Xiaoyang Liu
University of Waterloo x98liu@uwaterloo.ca

Abstract

We construct a Turing-complete computational model using monodromy and successive path lifting through covering spaces. The monodromy actions of five fixed loops produce memory shifts and conditional writes, while finite analytic path selection organizes these operations into programs. A Lean-verified simulation preserves complete Turing-machine configurations, termination, and divergence.

1 Introduction

A closed path can return to its base point while changing an analytic branch (e.g., continuing $\log z$ from $z = 1$ once counterclockwise around the origin returns to $z = 1$ with the value changing from 0 to $2\pi i$). The construction turns winding into a zero-conditional write by successive logarithmic lifts in one coupled covering, then combines these operations with finite analytic routing and classical counter encodings.

2 Geometric observation

Write $\mathbb{C}^* = \mathbb{C} \setminus \{0\}$ and $I = [0, 1]$. The exponential covering $\exp : \mathbb{C} \to \mathbb{C}^*$ has fiber $1 + 2\pi i\mathbb{Z}$ above $e = \exp(1)$. A fixed based loop and its lift from $1 + 2\pi ik$ are

$$b(t) = \exp(e^{2\pi it}), \tag{1}$$

$$w_k(t) = e^{2\pi it} + 2\pi ik. \tag{2}$$

Exponentiation gives (1), and the initial values agree. The curve w_0 winds once around zero. For $k \neq 0$, the circle w_k has radius 1 and centre at distance $2\pi|k| > 1$ from zero, so its winding number is zero. A second logarithmic lift therefore changes by $2\pi i$ exactly when $k = 0$.

3 The geometric construction

A shared upper root

A covering map and a point above the start of a path determine a unique lift. For analytic coverings, this continues a local inverse branch. Concatenation composes transport; reversal inverts it; endpoint-fixed homotopy preserves the returned value [1, §1.3].

To make both lifts admissible, set $Y = \mathbb{C} \setminus \{0, 1\}$ and $D = \exp^{-1}(Y)$. The restriction $\exp : D \to Y$ is a covering and $0 \notin D$. The loop b lies in Y: equality $b(t) = 1$ would give $e^{2\pi it} = 2\pi in$ for an integer n, contradicting its modulus. Thus the first lift avoids the puncture required for the second.

The two lower branches should contribute their winding to a shared upper branch. If ℓ_j continues a logarithm along w_j, then

$$e^{\ell_1+\ell_2} = w_1 w_2.$$

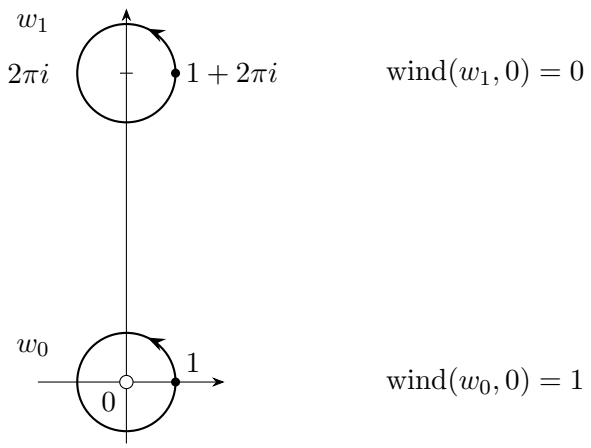


Figure 1: The lifted circles in the same punctured plane. They have equal radius and differ by $2\pi i$. Only w_0 surrounds zero, so only its logarithmic lift gains a turn.

Thus a logarithm of the product accumulates the two logarithmic increments additively. We use $w_1 w_2$ as the common upper input. To allow an independent upper turn, include a third base coordinate $v \in \mathbb{C}^*$ and impose

$$e^z = v\, w_1 w_2.$$

Solving for the recorded base value v gives the coupled projection

$$F(w_1, w_2, z) = \left(e^{w_1}, e^{w_2}, \frac{e^z}{w_1 w_2} \right) \tag{3}$$

from $E = (D \times D) \times \mathbb{C}$ to $X = (Y \times Y) \times \mathbb{C}^*$. It factors through $L = (D \times D) \times \mathbb{C}^*$ as

$$E \xrightarrow{p} L \xrightarrow{q} X, \tag{4}$$

where

$$p(w_1, w_2, z) = \left(w_1, w_2, \frac{e^z}{w_1 w_2} \right), \qquad q(w_1, w_2, v) = (e^{w_1}, e^{w_2}, v).$$

The map q is a product covering. The map p is the pullback of $\exp : \mathbb{C} \to \mathbb{C}^*$ along $(w_1, w_2, v) \mapsto v w_1 w_2$, with the determined coordinate v eliminated. A state is a retained point of $S = F^{-1}(e, e, 1)$. Put $r_k = 1 + 2\pi i k$. With the principal logarithm, this entire fiber is uniquely parametrized by

$$s(k_1, k_2, m) = \big(r_{k_1}, r_{k_2}, \operatorname{Log} r_{k_1} + \operatorname{Log} r_{k_2} + 2\pi i m \big), \qquad (k_1, k_2, m) \in \mathbb{Z}^3. \tag{5}$$

Indeed, the first two exponential equations give r_{k_1}, r_{k_2}, and the third gives precisely the remaining integer m.

Definition 3.1 (Successive execution)**.** A driver is a triple $\gamma = (u_1, u_2, v)$ of based loops at $(e, e, 1)$ in X. From $s = (w_1, w_2, z) \in S$, lift u_j through the exponential from w_j, obtaining $\widetilde{w}_j(t)$. Use the resulting path

$$\eta_s(t) = v(t) \widetilde{w}_1(t) \widetilde{w}_2(t) \tag{6}$$

as input to an exponential lift from z, obtaining $\widetilde{z}(t)$. Define $T_\gamma(s)$ to be the three endpoints.

The lower lifts remain in D, so (6) is nonzero. Its initial value is $w_1 w_2 = e^z$. The intermediate curve $\alpha_s = (\widetilde{w}_1, \widetilde{w}_2, v)$ projects by q to γ; the complete curve $\widetilde{\gamma}_s = (\widetilde{w}_1, \widetilde{w}_2, \widetilde{z})$ projects by p to α_s. Lift uniqueness identifies these curves with the two covering lifts in Figure 2. At the final time $e^{\widetilde{w}_j(1)} = e$ and $e^{\widetilde{z}(1)} = \widetilde{w}_1(1) \widetilde{w}_2(1)$, since $v(1) = 1$. The endpoint is therefore in the same fiber S, ready to serve as the next initial state.

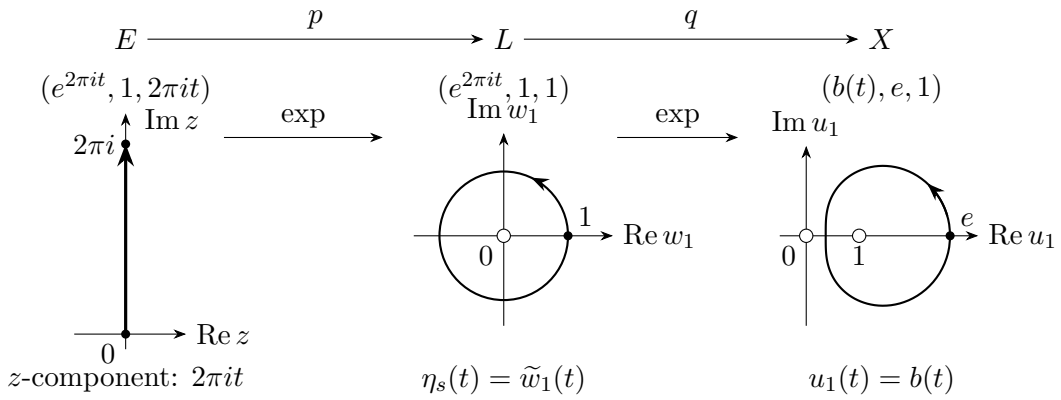


Figure 2: Actual transport for $\gamma = (b, e, 1)$ from $s(0, 0, 0)$. The tuples are the full curves; the component plots use separate scales. Read lifting from right to left. The component arrows show $\exp(\widetilde{z}) = \eta_s = \widetilde{w}_1$ and $\exp(\widetilde{w}_1) = b$. The right curve is drawn from $(\operatorname{Re} b, \operatorname{Im} b) = e^{\cos\theta}(\cos(\sin\theta), \sin(\sin\theta))$, $\theta = 2\pi t$. The upper lift is the straight segment $\widetilde{z}(t) = 2\pi it$.

Five fixed paths

Besides b, take $a(t) = \exp(1 + 2\pi it)$ and $c(t) = e^{2\pi it}$. Both a, b are based at e in Y; c is based at 1 in $\mathbb{C}^*$. Let A_j apply a to coordinate j, and B_j apply b there, with the remaining inputs constant. Let C apply c to the third base coordinate.

Proposition 3.2 (Native operations)**.** *The five actual transports act on the labels* (5) *by*

$$A_1 : (k_1, k_2, m) \longmapsto (k_1 + 1, k_2, m), \qquad A_2 : (k_1, k_2, m) \longmapsto (k_1, k_2 + 1, m),$$
$$B_1 : (k_1, k_2, m) \longmapsto (k_1, k_2, m + [k_1 = 0]), \qquad B_2 : (k_1, k_2, m) \longmapsto (k_1, k_2, m + [k_2 = 0]),$$
$$C : (k_1, k_2, m) \longmapsto (k_1, k_2, m + 1),$$

where $[P]$ *is* 1 *when* P *holds and* 0 *otherwise.*

Proof. For A_j, the lower lift is $1 + 2\pi i(k_j + t)$. Its real part is 1, so its principal logarithm is continuous. The upper lift changes by the difference of these logarithms, preserving m. For B_j, the lower lift is (2) and the other memory root is constant. Its winding calculation gives the conditional upper change. For C, both lower roots are constant and (6) winds once, so the upper lift adds $2\pi i$. Each displayed trajectory is identified with the actual lift by uniqueness. □

Products of drivers will be written in execution order. Concatenation gives $T_{\gamma\delta} = T_\delta \circ T_\gamma$, and path reversal gives $T_{\gamma^{-1}} = T_\gamma^{-1}$.

Example 3.3 (A complete path execution)**.** Execute the word $A_1B_1B_2A_2^{-1}B_2$ from $s(-1, 1, 0)$. The retained complex roots are

completed path	w_1	w_2	z
initial	r_{-1}	r_1	$\operatorname{Log} r_{-1} + \operatorname{Log} r_1$
A_1	1	r_1	$\operatorname{Log} r_1$
A_1B_1	1	r_1	$\operatorname{Log} r_1 + 2\pi i$
$A_1B_1B_2$	1	r_1	$\operatorname{Log} r_1 + 2\pi i$
$A_1B_1B_2A_2^{-1}$	1	1	$2\pi i$
$A_1B_1B_2A_2^{-1}B_2$	1	1	$4\pi i$

The first B_1 lifts to a circle about zero and writes one upper turn. The first B_2 lifts to a circle centred at $2\pi i$ and writes none. Reversing A_2 brings the second root from $1 + 2\pi i$ to 1; the last B_2 now writes a turn. Thus the final state is $s(0, 0, 2)$. Reversing the word recovers the initial state.

4 Executable programs

Selecting the next path

The conditional operation writes an upper branch. To let this branch choose the next path, fix $N \geq 1$ and put $\omega_j = e^{2\pi ij/N}$, $0 \leq j < N$. At a return, observe

$$\rho_N(w_1, w_2, z) = \exp\left(\frac{z - \operatorname{Log} w_1 - \operatorname{Log} w_2}{N}\right). \tag{7}$$

On S it equals $e^{2\pi im/N}$, an Nth root of 1. The logarithms are analytic near every return, since $\operatorname{Re} w_1 = \operatorname{Re} w_2 = 1$ there.

The normalized divided differences of u^N give the polynomials

$$K_j(u) = \frac{u^N - 1}{N\omega_j^{N-1}(u - \omega_j)}, \qquad K_j(\omega_j) = 1.$$

Their values satisfy $K_j(\omega_\ell) = [j = \ell]$. For a finite driver table γ_j, set, coordinatewise,

$$\gamma_s(t) = \sum_{j=0}^{N-1} K_j(\rho_N(s))\gamma_j(t). \tag{8}$$

The execution language admits (7) and (8) as finite analytic observation and path selection at exact fiber returns. On S they choose one admissible driver; N and the table are fixed for each program. The routing coefficients stay fixed during the word. Successive lifting supplies the conditional write; analytic routing chooses the next word.

Definition 4.1 (Finite geometric program)**.** A word is a finite concatenation of the five drivers and their reverses. A program P specifies $N \geq 1$, a word W_j and a bit h_j at each root ω_j. With γ_j the driver of W_j, its stopping signal is

$$H_P(s) = \sum_j K_j(\rho_N(s))h_j.$$

If $H_P(s) = 1$, execution stops at s; otherwise one return executes $T_{\gamma_s}(s)$. A program return is the completion of the selected word; its intermediate primitive endpoints do not trigger a new observation. The complete partial execution $\operatorname{ev}_P(s)$ is the first stopped state, and is undefined if returns continue indefinitely.

For example, choose A_1B_1 when $\rho_2 = 1$ and A_2B_1 when $\rho_2 = -1$. The successive states $s(-1, 0, 0) \mapsto s(0, 0, 1) \mapsto s(0, 1, 2)$ show how a conditional write changes the next selected memory operation. The roots, the finite table, and lifting determine every return.

Finite control and counters

Regard the two lower labels as nonnegative counters. A counter instruction increments counter i and jumps to q', or tests it, jumping to q_0 if zero and otherwise decrementing and jumping to q_1. Include a halt instruction.

Proposition 4.2 (Counter realization)**.** *Every finite nonempty two-counter table has a geometric program with the faithful encoding $(q, n_1, n_2) \mapsto s(n_1, n_2, 3q)$. Each live instruction takes exactly two native returns; halted instructions stop at their encoded states.*

Proof. For r addresses take $N = 3r$, reserving phases $3q, 3q + 1, 3q + 2$. An increment executes C first, then $A_iC^{3q'-3q-1}$. A conditional decrement executes CB_i first. The upper label becomes $3q + 2$ in the zero case and $3q + 1$ otherwise. The next word is respectively C^{3q_0-3q-2} or $A_i^{-1}C^{3q_1-3q-1}$. Here an integer power of C is the finite repetition of C or its reverse. Mark phase zero at halt addresses. Proposition 3.2 verifies these cases; finite root routing executes their words. □

5 Turing universality

The remaining construction translates finite tape computations into the counter instructions just realized:

$$\text{finite tapes} \longrightarrow \text{three counters} \longrightarrow \text{two counters} \longrightarrow \text{geometric words.}$$

A source machine has finite nonempty control Q, finite alphabet Γ with blank 0, and a partial instruction table $M : Q \times \Gamma \rightharpoonup Q \times (\{\mathsf{L}, \mathsf{R}\} \sqcup \Gamma)$. It moves the head or writes a symbol; an absent instruction halts. A configuration contains control, current symbol, and two blank-padded half-tapes with finite nonblank support. Write $\delta_M(c) = c'$ for one instruction step, and call c halted when no instruction applies. Let $\mathrm{ev}_M(c)$ be the final configuration when execution halts; it is undefined for an infinite run.

Theorem 5.1 (Native geometric Turing universality)**.** *Every such M has a finite geometric program P_M, an injective configuration encoding $\iota_M : \mathrm{Cfg}(M) \to S$, and a finite root-marked boundary. Its native macro $\mathcal{T}_M : S \rightharpoonup S \sqcup \{\mathsf{halt}\}$ reports* halt *when P_M's internal stopping signal is set; otherwise it executes one native word return by successive lifting, then continues the same word table until the next marked return. It is defined on every encoded configuration, with*

$$\mathcal{T}_M(\iota_M(c)) = \begin{cases} \iota_M(c'), & \text{if } \delta_M(c) = c', \\ \mathsf{halt}, & \text{if } c \text{ is halted.} \end{cases} \tag{9}$$

$$\mathrm{ev}_{P_M} \circ \iota_M = \iota_M \circ \mathrm{ev}_M\,. \tag{10}$$

Each live source step is a positive finite native first return. The partial maps in (10) *have the same domain: the configurations from which M halts. Their values encode the same complete final configuration, so divergence is preserved as well. Every finite source run is an actual successive lift through* (4)*, with base paths generated from preceding returned roots and endpoint the encoded final configuration.*

Proof. Encoding the tape. Enumerate $\Gamma = \{0, \dots, a-1\}$ with blank 0, and put $B = a + 1$. The canonical half-tape code $E(d_0 d_1 \cdots) = \sum_j d_j B^j$ ignores trailing blanks and is injective: successive remainders and quotients recover every cell. Pushing a symbol d gives $d + BE$; reading and removing it give $E \bmod B$ and $\lfloor E/B \rfloor$. Thus a finite controller storing the pair (control, current symbol), two counters storing the half-tapes, and a shared workspace counter can execute every tape instruction. The workspace is zero at each tape-step boundary.

Keeping two memory counters. To fit the two lower roots, compress these three counters by the classical prime encoding [2]

$$(r_0, r_1, r_2) \longmapsto U = 2^{r_0} 3^{r_1} 5^{r_2}.$$

The second target counter is workspace, again zero at boundaries. For $(p_0, p_1, p_2) = (2, 3, 5)$, increment of r_i multiplies by p_i; its test and decrement become divisibility and division by p_i. These are finite counter loops. Multiplication transfers units while adding p_i for each unit; division counts blocks of p_i, keeping the bounded remainder in finite control. A failed divisibility test restores the original value. Every such loop terminates and clears its workspace.

Executing the encoding. Proposition 4.2 compiles the resulting finite table into geometric words. If $d_M(q, h)$ is the three-counter dispatch address of (q, h) followed by its two-counter dispatch address, then

$$\iota_M(q, L, h, R) = s\big(2^{E(L)} 3^{E(R)},\, 0,\, 3 d_M(q, h)\big).$$

Injectivity follows from the tape codes, prime encoding, and dispatch maps. Fixing M fixes the address maps and the finite word table; c determines only the initial three roots.

Matching returns. Arithmetic loops occupy finite control regions disjoint from the tape-step dispatches. A second bit table b_j marks precisely $3d_M(q,h)$; replacing the halt bits h_j by b_j gives a boundary program with the same words. A live source step leaves one mark and reaches the next after finitely many words, with no intervening mark. This proves (9). Along these runs, the original halt bits stop exactly at halted source configurations. Iteration therefore gives (10): internal loops introduce neither extra results nor divergence. Concatenating the selected drivers from their actual returned states gives the two covering lifts and the encoded endpoint. The source transition appears in the proof of correspondence; path generation uses the fixed program and the preceding roots. □

6 Discussion and verification

Exponential fibers retain the memory; successive lifting supplies the conditional write; finite analytic root selection supplies instruction choice. Classical counter encodings organize these operations into universal computation. The covering maps and five primitives are common to all programs; the finite word table and its root degree depend on the source machine.

Analytic simulation of Turing machines has precedents in Koiran–Moore [3] and Graça–Campagnolo–Buescu [4], the latter including robustness. Cardona et al. construct a Turing-complete stationary Euler flow on a suitably metrized S^3 [5]. The present model executes at discrete returns by successive lifting and finite root selection. A single autonomous flow generating this execution would require a further construction. The semantics uses exact fiber states and return tests. Homotopy invariance preserves transport under admissible based deformations; quantitative error bounds, physical realization, and computational cost remain separate questions.

The integrated Lean 4 development verifies the construction and its simulation chain, using Mathlib's formal computability framework [6]. Appendix A gives a mathematical proof index and the commands for reproducing the checks.

AI contributions. Generative AI (OpenAI ChatGPT and Codex, including collaborating agents) contributed substantially to ideation, mathematical constructions, proofs, Lean formalization, implementation, and manuscript preparation. The human author directed the research, determined its scope and principles, critically evaluated the results, and assumes responsibility for the work. The source accompanies the manuscript for verification.

References

[1] A. Hatcher, *Algebraic Topology*, Cambridge University Press, 2002. `https://pi.math.cornell.edu/~hatcher/AT/ATpage.html`.

[2] M. L. Minsky, Recursive unsolvability of Post's problem of "tag" and other topics in theory of Turing machines, *Ann. of Math.* **74** (1961), 437–455.

[3] P. Koiran and C. Moore, Closed-form analytic maps in one and two dimensions can simulate universal Turing machines, *Theoret. Comput. Sci.* **210** (1999), 217–223. doi:10.1016/S0304-3975(98)00117-0.

[4] D. S. Graça, M. L. Campagnolo and J. Buescu, Robust simulations of Turing machines with analytic maps and flows, in *New Computational Paradigms*, LNCS **3526**, Springer, 2005, 169–179. doi:10.1007/11494645_21.

[5] R. Cardona, E. Miranda, D. Peralta-Salas and F. Presas, Constructing Turing complete Euler flows in dimension 3, *Proc. Natl. Acad. Sci. USA* **118** (2021), e2026818118. doi:10.1073/pnas.2026818118.

[6] M. Carneiro, Formalizing computability theory via partial recursive functions, *ITP 2019*, LIPIcs **141**, 12:1–12:17. doi:10.4230/LIPIcs.ITP.2019.12.

A Proof index and reproduction

The accompanying Lean 4.35.0 project supplies pinned dependencies. Its `README.md` records the Git base, exact source fingerprint, and proof index. Repository:
`https://github.com/ERHUTUZI123/monodromy`.
Paths below are relative to `src/Monodromy/`; declaration names begin with `Monodromy`. Short names retain the preceding namespace.

Conditional winding (Section 2). A fixed lower loop writes one upper turn precisely on the zero branch. The actual-lift calculation is `NestedContinuation.execute_prepare` in `Logarithm.lean`. The admissible lower covering is `NestedRestricted.firstCovering`. General transport and homotopy invariance are established in `Covering.lean`.

The shared fiber and successive execution (Definition 3.1). `Continuation.lean` proves that the state fiber has exactly the labels (5), that both maps in (4) are coverings, and that the complete runtime curves are their actual lifts. The principal declarations are `CoupledContinuation.prepare_surjective`, `prepare_injective`, `CoupledPullback.upperCovering`, `lowerCovering`, and `CoupledContinuation.restrictedTrajectory_actual_lift`.

Native paths (Proposition 3.2, Example 3.3). In `Continuation.lean`, namespace `CoupledContinuation`, the declarations `execute_A_left`, `execute_A_right`, `execute_B_left`, `execute_B_right`, and `execute_control_prepare` give the five return formulas. The worked word and its reverse are `example_run` and `example_reverse`.

Internal control (Definition 4.1). The finite-root kernels select a whole prescribed path. `Choice.lean` and `Radix.lean` establish their analytic and cardinal properties. Their connection to native execution is `CoupledContinuation.routeDriver_eq` and `Program.step_prepare` in `Continuation.lean`. This is the exact-return domain on which the routed paths are proved admissible.

Two-counter execution (Proposition 4.2). `CounterContinuation.two_returns` in `Counter.lean` verifies the two-return instruction implementation. `CounterContinuation.eval_encode` preserves its entire partial execution. The same file gives the tape and prime-compression reductions under `Counter.TuringCompiler` and `Counter.Compression`; canonical tape coding is in `Tape.lean`.

Native universality (Theorem 5.1). `NativeTuring.native_universality` in `FormalCompilation.lean` combines faithful configuration encoding, complete partial execution, exact first returns, and finite-run path realization. Its finite-type forms include `finiteMacro_iota` and `finite_eval_iota`; `result_actual_lifts` identifies both covering lifts and the final encoded root. Finite source renaming is in `FiniteSupport.lean`.

Reproduction. From the supplied Lean project root, run

```
lake exe cache get
lake build
lake env lean -DautoImplicit=false -DwarningAsError=true src/Audit.lean
```

The first command obtains the pinned external dependency cache; the build and audit pass for the supplied source snapshot. The audit examines every exported Monodromy declaration and its transitive proof dependencies, allowing only the standard axioms `propext`, `Classical.choice`, and `Quot.sound`.